NUTS: Learning Long-Horizon Cyclic Nut Threading via Hybrid-Teacher Policy Distillation

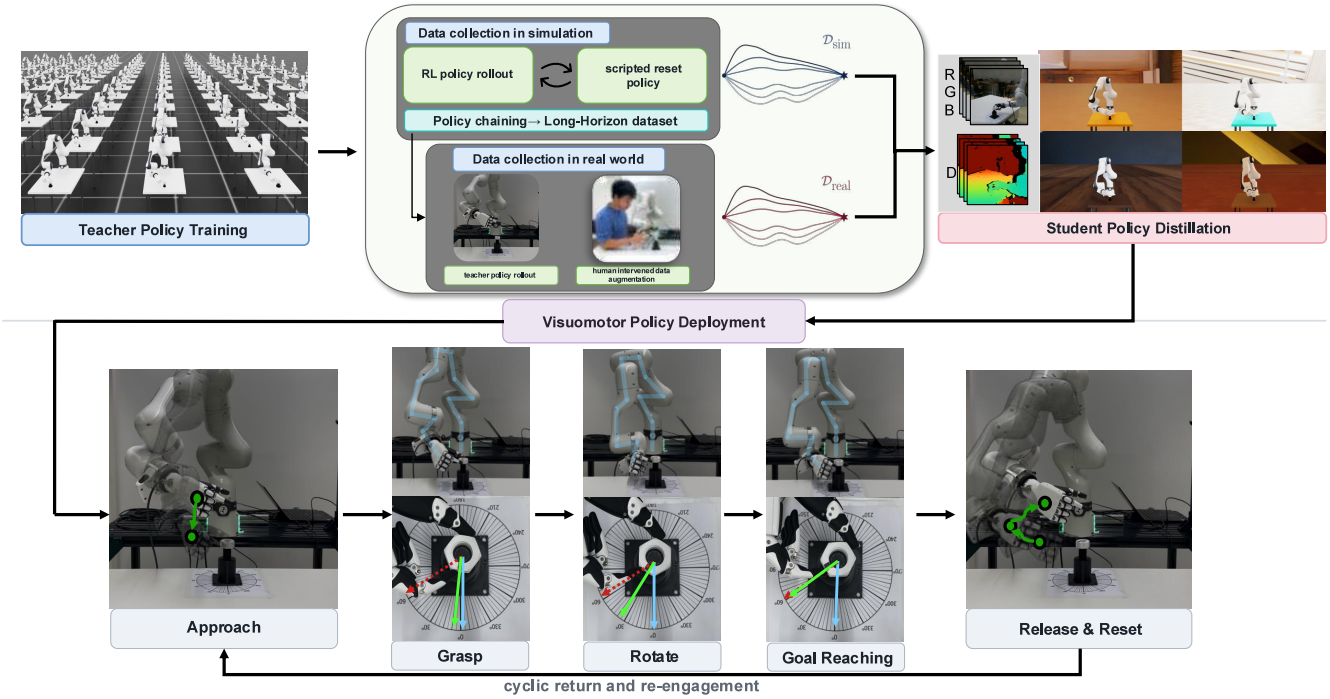


Fig. 1. **Overview of NUTS—decompose during data collection, unify at deployment.** During data collection, an RL policy executes approach, grasp, rotate, and goal reaching, while scripted controllers execute release and return&reset; together, they form a hybrid teacher for generating long-horizon threading demonstrations. Human corrections augment real-world replays to address the sim-to-real gap. Policy distillation then unifies these demonstrations into a single visuomotor policy that executes the complete cyclic task at deployment.

Abstract— **Learning controlled nut threading is one of the most challenging problems in robotic assembly. Repetitive phases—including grasping, turning, goal reaching, and release-and-reset—must transition seamlessly to sustain consecutive threading cycles, making the task long-horizon and sensitive to errors at phase boundaries. We found that learning all phases end to end with a single reinforcement-learning policy makes long-horizon reward shaping and training difficult. We therefore present NUTS, which follows a simple principle: decompose during data generation, unify during deployment. A hybrid teacher combines a privileged RL policy for approach through goal reaching with scripted release-and-reset behaviors to generate complete multi-cycle demonstrations. Sparse human corrections augment real-world replays to reduce the sim-to-real gap. We distill the combined data into a single conditional flow-matching visuomotor student that predicts eight-step action chunks from point-cloud features converted from RGB-D observations and executes the full task autonomously. We evaluate the student policy in 10-cycle trials using in-distribution M24, M30, and M36 hexagonal nuts and unseen rectangular, octagonal, and round nut geometries. For all in-distribution nuts, the policy completes all 10 cycles, with mean cyclic tracking errors below 6° in magnitude. Generalization is geometry dependent: the policy completes all 10 cycles on the round and octagonal profiles with comparable errors, but barely succeeds on rectangular profiles, indicating a limitation when test geometry does not support the learned grasping skill.**

I. Introduction

Threading a nut onto a bolt is among the most difficult tasks in robotic assembly because threaded engagement is notoriously contact-rich. On the one hand, the engaged threads can produce more than 15,000 simultaneous contact points, making faithful simulation prohibitively expensive for the large number of parallel rollouts required for reinforcement learning. Factory[1] directly addresses this computational bottleneck through a combination of signed-distance-field (SDF)[2] collision detection, contact reduction, and a Gauss–Seidel solver, enabling 1,000 parallel nut–bolt assemblies to be simulated in real time on a single GPU.

On the other hand, depending on the task, humans perform nut threading in various ways. For simple spinning, a single finger is sufficient while the wrist remains fixed. A pinch

grasp is preferred for controlled threading, whereas a tripod or even a power grasp may be applied in high-resistance scenarios.

Prior work has typically approached the problem from the perspective of spinning. DexScrew[3] uses a dexterous hand–arm setup similar to ours and replaces explicit thread geometry with a revolute-joint proxy to learn a rotational finger gait. The learned skill then assists teleoperation and supports the collection of real-world demonstrations for tactile behavior cloning. However, its policy performs uncontrolled spinning without target conditioning or learned termination, leaving goal-directed, cyclic nut threading unresolved.

Play2Perfect[4] builds on the goal-conditioned object-manipulation policy introduced by SimToolReal[5]. It pre-trains this policy through play on diverse objects and goals, then fine-tunes it for precise assembly tasks, including screwing a rod-like furniture leg. This formulation enables goal-directed screwing in a structured setting but does not address everyday threaded objects, such as nuts and bolts.

We first attempted to approach the controlled threading problem for nuts and bolts by training a unified teacher policy for the complete multi-cycle task. In practice, however, designing rewards that capture both long-horizon progress and precise transitions between phases proved exceedingly difficult.

This motivates our central design principle: **decompose during data generation, unify during deployment**. During data generation, a learned policy handles approach, grasp, rotate, and goal reaching, while scripted transition controllers handle release-and-reset behaviors. We refer to the composition of the learned RL skill and scripted transition controllers as the hybrid teacher, while human intervention is used only during real-world replay to correct teacher trajectories under sim-to-real discrepancies. During deployment, a student policy distills these heterogeneous demonstrations into a single visuomotor policy that executes the complete cyclic task without privileged state, scripted controllers, or human intervention. We evaluate threading accuracy in both simulation and real-world experiments.

The main contributions of this work are:

- A long-horizon cyclic nut-threading formulation and hybrid teacher that combines an RL policy for approach, grasp, rotation, and goal reaching with scripted release-and-reset behaviors, generating coherent long-horizon, multi-cycle demonstrations without a monolithic RL teacher.
- A human-corrected sim-to-real demonstration strategy that selectively replaces failure-prone segments during real-world replay, adapting simulated behaviors to hardware contact and perception discrepancies.
- A unified 3D flow-matching visuomotor student that distills heterogeneous demonstrations into complete cyclic behavior without privileged state, scripted controllers, or human assistance at deployment. In real-world tests, it completes every 10-cycle trial on all training sizes and geometrically compatible unseen round and octagonal profiles, while exposing failure on the tested rectangular profiles.

II. Related Work

Robot Assembly with Dexterous Manipulation. Robot assembly requires precise motion and closed-loop contact under geometric and dynamic uncertainty [6], with tasks including peg-in-hole insertion, gear meshing, connector mating, screwdriving, and nut–bolt fastening. Classical systems rely on fixtures or specialized tools, whereas multi-fingered hands offer greater versatility but introduce high-dimensional control and complex contact dynamics. Learning-based systems include Factory[1] for scalable simulation, FORGE[7] for force-guided contact search, SimToolReal[5] for dexterous tool use, and Play2Perfect[4] for reusable manipulation priors. Closest to our work, DexScrew[3] learns sustained rotation of an already engaged fastener from simplified simulation. In contrast, NUTS performs controlled, multi-cycle nut threading with coordinated turning and release-and-reset behavior across multiple nut sizes.

Policy Chaining in Long-Horizon Tasks. Training a single policy for a long-horizon task is challenging: small control errors can compound over time [8], while sequential fine-tuning on new subtasks can overwrite previously acquired behaviors and cause catastrophic forgetting [9]. Common remedies include increasing network capacity, freezing or regularizing task-relevant parameters, and adopting architectures with stronger temporal memory, such as recurrent networks or Transformers. An alternative is to decompose the task into reusable skills and chain their policies. This decomposition reduces the burden on each individual policy but introduces a transition problem: the terminal states produced by one skill may fall outside the initial-state distribution of the next, causing failures at skill boundaries. Adversarial Skill Chaining[10] regularizes terminal states to reduce this mismatch. Sequential Dexterity[11] addresses it using a learned transition-feasibility function that fine-tunes adjacent dexterous policies, supports autonomous switching, and enables recovery.

Sim-to-Real Adaptation with Human Intervention. The sim-to-real gap is a long-standing problem in robot learning: policies trained in simulation encounter mismatches in perception, dynamics, contact, and control when deployed on physical hardware. These discrepancies become increasingly consequential for high-dimensional, contact-rich dexterous manipulation. Most approaches seek robust zero-shot transfer through domain randomization [12] or domain adaptation through Adapt3R[13], while methods such as TRANSIC[14] use limited real-world data to adapt a simulation-trained policy. DAgger[8] aggregates expert labels on states visited by the learner, whereas SafeDAgger[15] reduces expert queries by requesting supervision only when the learned policy is predicted to be unsafe. Similarly, during real-world hybrid-teacher replay, we replace anticipated slip or misalignment failures with human corrections and add the corrected trajectories to the student-distillation dataset.

Visuomotor Policy Distillation. Learning by Cheating[16] demonstrates a practical route from policies trained with

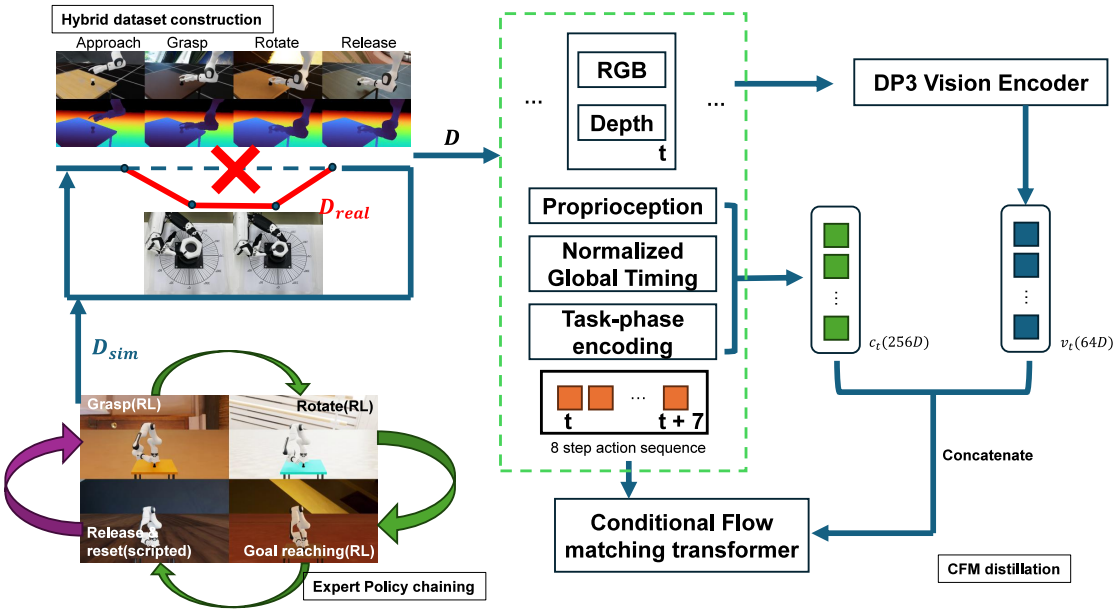


Fig. 2. Detailed NUTS pipeline. An RL teacher is repeatedly rolled out with a scripted policy to construct cyclic threading demonstrations in simulation; together, they form the hybrid teacher. During real-world replay, human corrections replace flawed trajectory segments from grasp through release, forming a mixed simulation and real-world dataset. The conditional flow-matching student takes a 64-D visual feature extracted by a DP3 vision encoder from RGB-D observations and a 256-D state feature comprising proprioception, timing, and phase encodings, and predicts an eight-step action chunk.

privileged simulator state to deployable policies that act from onboard sensing. In dexterous manipulation, DextrAH-RGB[17] distills a privileged reinforcement-learning teacher into an end-to-end visual grasping policy, while UniGrasp-Transformer[18] consolidates object-specific teacher policies into a unified student. For long-horizon tasks, predicting action sequences improves temporal consistency, while generative models can represent multiple valid behaviors. Diffusion Policy[19] models multimodal action sequences through iterative denoising. Flow Matching[20] instead learns a continuous vector field for efficient ODE-based action generation, as adopted by π_0[21]. These properties motivate its use for distilling our hybrid-teacher and human-corrected demonstrations.

III. Problem Statement and Method

A. Problem Formulation

We decompose cyclic nut threading into the ordered phase sequence

$$\mathcal{S} = \big(\underbrace{\text{approach}, \text{grasp}, \text{rotate}, \text{goal reaching}}_{\mathcal{M}_{\text{RL}}:\, \pi_T}, \underbrace{\text{release}, \text{return\&reset}}_{\mathcal{M}_{\text{script}}:\, \boldsymbol{\kappa}}\big). \tag{1}$$

Thus, the active phase satisfies $m_t \in \mathcal{M} = \mathcal{M}_{\text{RL}} \cup \mathcal{M}_{\text{script}}$. The RL-learned teacher π_T executes the four phases in $\mathcal{M}_{\text{RL}}$, whereas scripted controllers $\boldsymbol{\kappa}$ execute the release-and-reset phases in $\mathcal{M}_{\text{script}}$.

Let K be the predefined cycle count, $\Delta\varphi^*$ the desired clockwise rotation per cycle, and φ_k the unwrapped nut angle upon completing the goal-reaching phase in cycle k. We define the cycle and episodic tracking errors as

$$\begin{aligned} e_k &= \varphi_k - \varphi_{k-1} - \Delta\varphi^*, \\ e_{\text{ep}} &= \varphi_K - \varphi_0 - K\Delta\varphi^* = \sum_{k=1}^{K} e_k, \\ \mathcal{E}(\tau) &= \frac{1}{K}\sum_{k=1}^{K} e_k^2 + \beta e_{\text{ep}}^2. \end{aligned} \tag{2}$$

Positive (negative) errors indicate overshoot (lag); $\mathcal{E}$ squares them to prevent cancellation.

Our research problem is then to find the policy that chains all phases with maximum completion success and minimum tracking error over the episode. Let $\Pi_{\mathcal{S}}$ denote the policies that execute $\mathcal{S}$ in order in every cycle. We express the two priorities as

$$\begin{aligned} \Pi_{\text{succ}} &= \arg\max_{\pi\in\Pi_{\mathcal{S}}} \Pr_{\tau\sim p_\pi}[C(\tau) = K], \\ \pi^* &= \arg\min_{\pi\in\Pi_{\text{succ}}} \mathbb{E}_{\tau\sim p_\pi}[\mathcal{E}(\tau) \mid C(\tau) = K]. \end{aligned} \tag{3}$$

Here, $C(\tau)$ counts completed cycles. Equation (3) first maximizes K-cycle success and then minimizes tracking cost, with β weighting accumulated drift relative to cycle-wise accuracy. We set $K = 10$ and $\Delta\varphi^* = 60°$, and learn π^* from fixed hybrid-teacher demonstrations rather than through on-hardware optimization.

B. Overview of the Distillation Pipeline

The proposed pipeline, illustrated in Fig. 2, comprises three stages. First, we train a privileged RL policy for approach,

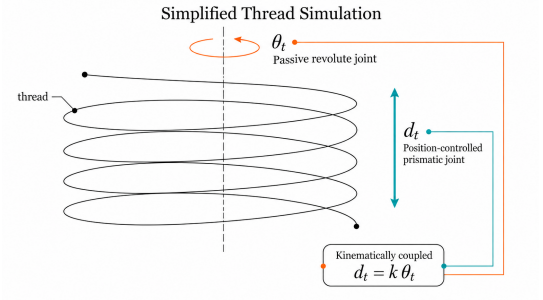


Fig. 3. Coupled revolute–prismatic thread model, with $k = -\rho/(2\pi)$ in (4).

grasp, rotate, and goal reaching and combine it with scripted release-and-reset controllers to construct a hybrid teacher. Second, we use the hybrid teacher to generate complete cyclic demonstrations in simulation and replay the resulting trajectories on the real system. A human operator intervenes whenever a grasp failure is anticipated. We consolidate the simulated and human-corrected real-world demonstrations into a fixed offline dataset. Third, we convert the student's RGB-D observations into point clouds, encode them with the proprioceptive observations, and use conditional flow matching to distill the demonstrated action-chunk distribution into a single visuomotor policy.

C. Privileged Teacher Policy Training

1) Simplified Thread Simulation: Our simplified thread model couples a passive revolute joint to a coaxial, position-controlled prismatic joint, avoiding explicit thread contacts, as illustrated in Fig. 3. With prismatic coordinate d_t, unwrapped direction-corrected revolute coordinate θ_t, thread pitch ρ, and turn-start index b_k, the prismatic target is

$$d_t^* - d_{b_k} = -\frac{\rho}{2\pi}\left(\theta_t - \theta_{b_k}\right), \tag{4}$$

so one revolution produces one thread pitch of axial travel. Finger–nut contact forces remain active, while the revolute–prismatic mechanism constrains lateral motion and keeps the nut near the commanded helix. The finite axial-drive stiffness permits small deviations from this path. For revolute-joint velocity ω_t, smoothed Coulomb and viscous friction give $\bar{\tau}_{f,t} = -\tau_c \tanh(\omega_t/\omega_s) - c_v\omega_t$, where τ_c is the Coulomb friction torque, c_v is the viscous damping coefficient, and ω_s is the smoothing speed.

2) PPO Training: We train the privileged teacher using an asymmetric recurrent actor–critic. The actor input contains noisy fingertip pose, fingertip velocity, nut-relative grasp state, turning-target error, a grasp-phase flag, the previous action, and a nut-size one-hot code. The critic follows the same task and history structure using clean simulator values, and extends it with arm-joint positions, complete nut–bolt geometry, controller state, and wrist and fingertip contact state. The one-hot code conditions a single teacher on M24, M30, and M36 hexagonal nuts. The actor and central critic each use a two-layer LSTM with 1024 hidden units per layer, followed by a 512–128–64 ELU MLP; recurrent sequences span 128 control steps. The actor parameterizes a diagonal Gaussian over the normalized action $\mathbf{a}_t \in [-1, 1]^9$.

We use a 9-D action space comprising a 6-D Cartesian controller target native to the Franka arm and three finger-control dimensions.

The resulting recurrent policy $\pi_T(\mathbf{a} \mid \mathbf{z}^\pi, \mathbf{h})$ is optimized with PPO using the shaped reward

$$\begin{aligned} r_t &= \sum_{i\in\mathcal{R}} w_i^r R_{i,t} - \sum_{j\in\mathcal{P}} w_j^p P_{j,t}, \\ \mathcal{R} &= \{\text{center}, \text{face}, \text{grasp}, \text{turn}, \text{target}\}, \\ \mathcal{P} &= \{\text{mag}, \Delta a, \text{align}, \text{force}, \text{settle}, \text{clear}, \text{contact}\}. \end{aligned} \tag{5}$$

The reward follows three stages. During approach, R_t^{center} rewards motion toward the nut center and R_t^{face} rewards alignment with the designated opposing faces. R_t^{grasp} marks acquisition of a stable, proper grasp. During turning, R_t^{turn} rewards grip-maintaining progress toward the fixed helical target. At completion, R_t^{target} rewards holding the target stably. The approach and turning rewards pay only new-best progress, preventing repeated motion from accumulating reward.

The penalties discourage large commands (P_t^{mag}), abrupt action changes ($P_t^{\Delta a}$), hand–nut misalignment (P_t^{align}), excess wrist force (P_t^{force}), residual rotation near the target (P_t^{settle}), insufficient table clearance (P_t^{clear}), and arm–table contact (P_t^{contact}).

We randomize the nut's initial rotation and axial height and the controller settings to improve teacher robustness.

D. Mixed-Dataset Construction with Human-Corrected Real-World Augmentation

1) Cyclic Teacher Rollout in Simulation: In simulation, we combine the learned RL policy for $\mathcal{M}_{\text{RL}}$ with scripted controllers for $\mathcal{M}_{\text{script}}$. Let $m_t \in \mathcal{M}$ denote the active phase, $\mathbf{x}_t$ the full simulator state, and $\mathbf{z}_t^\pi = g_\pi(\mathbf{x}_t, \boldsymbol{\xi}_t)$ the actor observation obtained by selecting the actor-visible state components and applying observation noise $\boldsymbol{\xi}_t$; $\mathbf{h}_t$ is the actor's recurrent hidden state. During rollout, we use the deterministic mean action of the Gaussian teacher policy, $\bar{\pi}_T(\mathbf{z}_t^\pi, \mathbf{h}_t) = \mathbb{E}_{\mathbf{a}\sim\pi_T(\cdot|\mathbf{z}_t^\pi,\mathbf{h}_t)}[\mathbf{a}]$. The hybrid teacher is

$$\pi_{\text{HT}}(\mathbf{x}_t, m_t) = \begin{cases} \bar{\pi}_T(\mathbf{z}_t^\pi, \mathbf{h}_t), & m_t \in \mathcal{M}_{\text{RL}}, \\ \boldsymbol{\kappa}_{m_t}(\mathbf{x}_t), & m_t \in \mathcal{M}_{\text{script}}, \end{cases} \tag{6}$$

where $\boldsymbol{\kappa}_{m_t}$ is the scripted controller for the active transition phase; both branches use the same filtered, bolt-relative Cartesian controller action space.

Release is triggered when the rotation lies within $\pm 5^\circ$ of φ_{tar} (60° in our experiments), while a valid opposed-finger grasp is maintained for a short dwell period. The fingers then open to a safe clearance, after which scripted arm–hand waypoints return to the initial grasp pose and re-engage the nut, completing the release-and-reset behavior. The turning reference is then updated, but the physical nut pose is not reset, producing complete multi-cycle demonstrations.

2) Data Augmentation under Human Intervention: We replay simulated trajectories on the real system through the same Cartesian controller. When slip or misalignment is anticipated, an operator replaces the affected teacher-action segment from grasp through release, recovering stable contact and completing the turn. These human actions overwrite the corresponding teacher actions in the trajectory, and the resulting corrected trajectory is recorded as a real-world demonstration.

Before training, we combine simulated rollouts and human-corrected replays into a fixed offline dataset of RGB-D observations, proprioception, normalized task progress, phase encodings, and executed commands. We mask chunks that cross physical resets, retain phase boundaries, and pad terminal chunks with the final valid command.

E. Student Policy Distillation

Student learning is posed as supervised imitation from the fixed dataset of simulated hybrid-teacher and human-corrected demonstrations, $\mathcal{D} = \mathcal{D}_{\text{sim}} \cup \mathcal{D}_{\text{real}}$. At time t, the student observes $\mathbf{y}_t = (\mathbf{O}_t^{\text{RGBD}}, \mathbf{s}_t, p_t, \mathbf{e}_{m_t})$ and learns the distribution of the executed H-step action chunk, where $\mathbf{e}_{m_t}$ is the one-hot encoding of the active phase $m_t \in \mathcal{M}$ from (1).

Following DP3[22], we transform each RGB-D observation into a point cloud $\mathcal{P}_t$ for further processing. A compact PointNet with 64–128–256 pointwise layers and global max pooling encodes $\mathcal{P}_t$ into a 64-D feature $\mathbf{v}_t$, which is fused with proprioception, task progress, and phase information. A 128–128 ELU MLP encodes proprioception, and fusion yields the 320-D condition $\mathbf{c}_t$. With $H = 8$, a four-layer non-causal Transformer (512-D width, eight heads, and a 2048-D GELU MLP) predicts velocities for the eight 9-D action tokens, conditioned on $\mathbf{c}_t$ and a sinusoidal flow-time embedding. During distillation, we follow DEXTRAH-RGB[17]-style lighting and texture randomization to improve robustness and reduce the sim-to-real gap.

The supervision target is the executed action chunk $\mathbf{U}_t^D = [\mathbf{u}_t^D, \ldots, \mathbf{u}_{t+H-1}^D] \in \mathbb{R}^{H\times 9}$. We formulate its distillation as conditional flow matching:

$$\begin{aligned} \theta^* = \arg\min_{\theta} \quad & \mathbb{E}\left[\left\| v_\theta(\mathbf{U}_t^\lambda, \lambda \mid \mathbf{c}_t) - \dot{\mathbf{U}}_t^\lambda \right\|_2^2\right], \\ \text{s.t.} \quad & \mathbf{U}_t^\lambda = (1-\lambda)\mathbf{U}_t^0 + \lambda \mathbf{U}_t^D, \\ & \dot{\mathbf{U}}_t^\lambda = \mathbf{U}_t^D - \mathbf{U}_t^0. \end{aligned} \tag{7}$$

The expectation is over $(\mathbf{y}_t, \mathbf{U}_t^D) \sim \mathcal{D}$, $\mathbf{U}_t^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and $\lambda \sim \mathcal{U}[0,1]$. The condition $\mathbf{y}_t$ contains RGB-D, proprioception, task progress, and phase information and is encoded as $\mathbf{c}_t$. The learned velocity field transports Gaussian noise to the conditional action distribution $p_\theta(\mathbf{U}_t \mid \mathbf{y}_t)$.

Algorithm 1 Offline flow distillation with human intervention

Require: Hybrid teacher π_{HT}, human operator π_H, horizon H

1: Collect simulated rollouts $\mathcal{D}_{\text{sim}}$ using π_{HT}
2: Replay them in the real world, overwriting failure-prone grasp-to-release segments with π_H to obtain $\mathcal{D}_{\text{real}}$
3: Form H-step chunks from $\mathcal{D} = \mathcal{D}_{\text{sim}} \cup \mathcal{D}_{\text{real}}$
4: **repeat**
5: Sample $(\mathbf{y}_t, \mathbf{U}_t^D) \sim \mathcal{D}$, $\mathbf{U}_t^0 \sim \mathcal{N}(0, I)$, and $\lambda \sim \mathcal{U}[0,1]$
6: Encode the point cloud and fuse it with the conditioning inputs to obtain $\mathbf{c}_t$
7: Update θ by minimizing (7)
8: Update EMA parameters $\bar{\theta} \leftarrow \alpha\bar{\theta} + (1-\alpha)\theta$
9: **until** convergence
10: **return** Student velocity field $v_{\bar{\theta}}$

IV. Experiments

A. Evaluation Metrics

We evaluate each condition over $N = 10$ trials, with up to ten cycles per trial. Each cycle consists of a 60° clockwise turn followed by the release-and-reset behavior.

a) Mean Completed Cycle Count: For trial i, $C_i \in \{0, 1, \ldots, 10\}$ counts completed cycles before failure or timeout; incomplete cycles are excluded. We report $\bar{C} = N^{-1}\sum_i C_i$; higher is better.

b) Cyclic Tracking Error: For completed cycle k in trial i, the angular error is $e_{i,k} = \Delta\varphi_{i,k} - 60^\circ$, where $\Delta\varphi_{i,k} = \varphi_{i,k} - \varphi_{i,k-1}$ is the clockwise nut rotation. Over all $M = \sum_i C_i$ completed cycles, we report $\mu_{\text{cyc}} = M^{-1}\sum_{i,k} e_{i,k}$ and standard deviation σ_{cyc}. Negative values indicate lag and positive values overshoot; a mean closer to zero and a smaller deviation are better. If $M = 0$, both cyclic-error statistics are reported as "–."

c) Ten-Cycle Tracking Error: Let $\mathcal{I}_{10} = \{i : C_i = 10\}$ denote the trials that complete all ten cycles. For each $i \in \mathcal{I}_{10}$, the ten-cycle error is $e_{10,i} = \sum_{k=1}^{10} \Delta\varphi_{i,k} - 600^\circ$. We report $\mu_{10} = |\mathcal{I}_{10}|^{-1}\sum_{i\in\mathcal{I}_{10}} e_{10,i}$ and its standard deviation over completed ten-cycle trials. Trials with fewer than ten completed cycles are excluded; if $\mathcal{I}_{10}$ is empty, the ten-cycle error is reported as "–." All errors are in degrees and use the sign convention above.

B. Experimental Setup

We simulate a seven-DoF Franka Research 3 (FR3) arm equipped with an Inspire Hand RH56DFX using Isaac Sim 5.1.0 and Isaac Lab 2.2.1. The 9-D action space encodes a six-DoF Cartesian-controller target and three index–thumb joint commands. At 15 Hz, the student converts each RGB-D observation into a point cloud for further processing and fuses a 64-dimensional DP3 feature with proprioception, progress, and phase; physics runs at 120 Hz. For the retained M24 run, we train the PPO teacher for 200 epochs in 512 environments. Using only 20 successful ten-cycle demonstrations (19,486 frames) collected from 32 environments, we increase trajectory diversity by applying 30 random arm impulses to each demonstration. On a single NVIDIA GeForce RTX 5090, training one teacher policy takes approximately 10 hours, while student-policy training takes approximately 30 minutes.

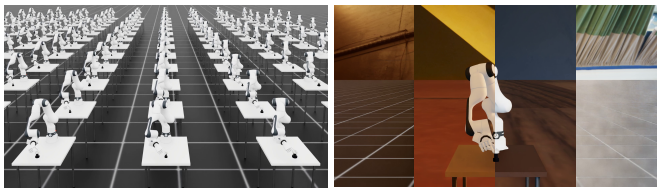


Fig. 4. Teacher policy training in Isaac Sim (left) and rollouts with lighting and texture randomization (right).

For real-world augmentation, 12 of the 20 replays (60%) contain at least one human correction. Human-overwritten grasp-to-release segments comprise only 6.48% of replay frames; the remaining 93.52% retain hybrid-teacher actions, showing that intervention is temporally sparse. We train the flow-matching student for 300 epochs using eight-step action chunks.

The physical platform comprises the same FR3 arm and Inspire Hand RH56DFX. An Intel RealSense D415 supplies the RGB-D observations, which are converted into point clouds before student inference. Robot commands use the same Cartesian controller as in simulation. In real-world deployment, the flow-matching policy uses a prediction horizon of eight and 16 ODE integration steps. Teacher and student training use only hexagonal nuts. We test M24, M30, and M36 hexagonal nuts in distribution and hold out the octagonal, rectangular, and round geometries of each size for out-of-distribution evaluation.

C. Ablations

To investigate the contribution of individual components in our framework, we conduct ablation studies focusing on four key aspects: **(Q1)** whether a unified multi-cycle student outperforms repeated execution of a single-cycle policy, **(Q2)** the contribution of the DP3 point-cloud encoder, **(Q3)** the effectiveness of flow matching relative to deterministic behavior cloning with LSTM and Transformer backbones, and **(Q4)** the benefit of augmenting simulation demonstrations with human-corrected real-world trajectories during student distillation. Results are averaged over 10 trials on the M24 hexagonal nut and reported in Table I.

We compare six ablations under matched training and evaluation budgets, changing only the component specified by each ablation.

(a) Single-Cycle Student. We train on real-world-augmented single-cycle demonstrations and invoke the resulting policy ten times, resetting its temporal context after each cycle.

(b) Ours + ResNet Visual Encoder. We replace DP3 with a one-branch, ImageNet-pretrained ResNet-18 stage-2 encoder that fuses RGB, log depth, validity, and calibrated XYZ geometry into a 64-D feature, while keeping the policy, objective, data, and training budget fixed.

(c) DP3 Vision + LSTM Distillation. Using the same real-world-augmented dataset as ours, we replace the flow-matching Transformer with the DEXTRAH-RGB[17]-style backbone—a layer-normalized 512-unit LSTM followed by a 512–512–256 ELU MLP—trained using deterministic supervised action regression.

(d) DP3 Vision + Transformer Distillation. Inspired by the Transformer-based action-chunking design of ACT[23], we use a deterministic variant without the CVAE objective. It has four layers, eight attention heads, a 512-D hidden representation, 2048-D feed-forward layers, and eight learned action queries that predict an eight-step action chunk.

(e) Ours with Only Simulated Data. We train the multi-cycle student using only simulated hybrid-teacher demonstrations, excluding all human-corrected real-world data.

(f) Ours. We train one flow-matching student on complete multi-cycle hybrid-teacher demonstrations augmented with human-corrected real-world data.

D. Results

TABLE I: Simulation and real-world ablations on the M24 hexagonal nut over 10 trials. Each defined entry reports the mean ± standard deviation. Completed-cycle count is averaged across trials, cyclic-error statistics use successful cycles only, and ten-cycle error compares accumulated rotation with the 600° target. Ten-cycle error is reported as "–" when no trial completes all 10 cycles. Errors are signed in degrees: negative denotes lagging and positive denotes overshooting.

(a) Simulation

Ablation	Completed cycles ↑	Cyclic tracking error → 0	10-cycle tracking error → 0
(a)	5.5 ± 0.8	-4.16 ± 3.21	–
(b)	2.3 ± 0.9	$+20.17 \pm 7.80$	–
(c)	0.0 ± 0.0	–	–
(d)	1.4 ± 0.8	$+124.21 \pm 18.50$	–
(e)	10.0 ± 0.0	-1.77 ± 1.02	-17.7 ± 3.23
(f) Ours	10.0 ± 0.0	-2.13 ± 1.10	-21.30 ± 3.48

(b) Real World

Ablation	Completed cycles ↑	Cyclic tracking error → 0	10-cycle tracking error → 0
(a)	5.2 ± 1.2	-4.56 ± 5.55	–
(b)	1.1 ± 0.9	$+24.09 \pm 9.94$	–
(e)	2.1 ± 1.4	-10.05 ± 4.08	–
(f) Ours	10.0 ± 0.0	-4.21 ± 1.20	-42.10 ± 4.37

To address **Q1**, we compare the single-cycle student (a) with our multi-cycle student (f). Our full method completes all 10 cycles in both simulation and the real world, whereas the single-cycle student completes only 5.5 and 5.2 cycles on average, respectively. A typical failure mode is that the single-cycle-trained student slips off the edge of the nut after a few cycles, showing that (a) learns the basic threading skill but fails to follow the axial motion of the nut (Table I).

For **Q2**, we perform an ablation on the vision encoder using both DP3 and ResNet-18 backbones, while holding the policy, objective, data, and training budget fixed. A shared decoder is used to evaluate the prediction error of the two encoded features. The DP3 encoder achieves a positional error of 8 mm and a rotational error of 15.16° in simulation, while ResNet-18 achieves a similar positional error of 8.6 mm but a notably higher rotational error of 31.20°. Real-world conditions further amplify the difference, with ablation (f) completing all 10 cycles while ablation (b) completes only 1.1 cycles on average. The substantially larger rotational

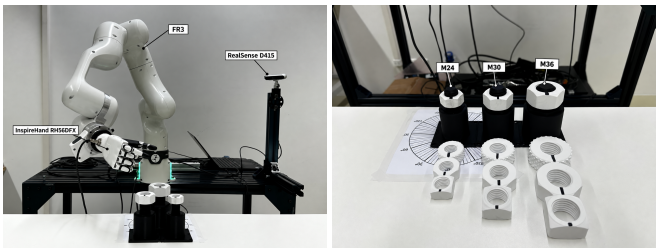


Fig. 5. Real-world setup (left) and test-object family (right), comprising M24, M30, and M36 hexagonal, octagonal, rectangular, and round nuts with corresponding bolts and holders.

prediction error of the ResNet representation suggests that image-space features are less effective at preserving the geometry required for repeated re-grasping under viewpoint and axial changes (Table I).

Results from ablations (c), (d), and (f) show that the choice of student action backbone makes a substantial difference in this long-horizon task, answering **Q3**. While the LSTM backbone fails completely in simulation, diverging before completing even the first cycle, the Transformer backbone survives slightly longer and completes 1.4 cycles on average, but soon faces the same compounding-error problem. Neither ablation (c) nor (d) advances to real-world testing (Table I).

The comparison between ablations (e) and (f) provides strong evidence for **Q4**. As shown in Table I, the student policy distilled using only simulation data performs perfectly in Isaac Sim, completing all 10 cycles and even achieving a lower cyclic tracking error and standard deviation than our full method. However, when deployed in the real world, the hardware sim-to-real gap greatly affects its threading success rate, leaving ablation (e) able to complete only 2.1 cycles on average, with a cyclic tracking error of -10.05° and a standard deviation of 4.08°. In contrast, our full method maintains consistent performance in both simulation and real-world tests, completing all 10 cycles. In the real world, it achieves a cyclic tracking error of -4.21° with a standard deviation of 1.20°, slightly higher than in simulation.

TABLE II: Geometry-dependent real-world transfer across nut sizes and profiles over 10 trials per nut. Values are mean ± standard deviation; shaded rows denote out-of-distribution geometries.

Size	Type	Completed cycles ↑	Cyclic tracking error → 0	10-cycle tracking error → 0
M24	Hex.	10.0 ± 0.0	-4.21 ± 1.20	-42.10 ± 4.37
	Oct.	10.0 ± 0.0	-4.10 ± 1.16	-41.00 ± 4.20
	Rect.	3.0 ± 1.2	$+30.30 \pm 4.50$	–
	Round	10.0 ± 0.0	-2.33 ± 0.91	-23.30 ± 3.58
M30	Hex.	10.0 ± 0.0	-3.70 ± 0.77	-37.00 ± 2.37
	Oct.	10.0 ± 0.0	-3.66 ± 0.58	-36.60 ± 1.74
	Rect.	0.0 ± 0.0	–	–
	Round	10.0 ± 0.0	-2.45 ± 0.49	-24.50 ± 1.43
M36	Hex.	10.0 ± 0.0	-5.52 ± 1.34	-55.20 ± 5.44
	Oct.	10.0 ± 0.0	-5.48 ± 1.22	-54.80 ± 4.75
	Rect.	0.0 ± 0.0	–	–
	Round	10.0 ± 0.0	-4.78 ± 1.02	-47.80 ± 2.36

In Table II, we provide test data for the in-distribution M24, M30, and M36 hexagonal nuts, as well as the out-of-distribution octagonal, rectangular, and round nuts, to characterize geometry-dependent transfer. Our method completes the task on the in-distribution training objects. The M30 nut achieves the smallest lag, with an average of 3.7° per cycle and 37° over the ten-cycle trial. The M36 hexagonal nut has the largest lag among the in-distribution sizes. We hypothesize that its larger geometry increases sensitivity to simulation-model errors, including penetration-depth mismatch; however, our experiments do not isolate this factor.

Interestingly, among the tested out-of-distribution nut shapes, the round nut achieves the best performance, followed by the octagonal nut, in terms of both mean cyclic tracking error and ten-cycle tracking error. Because we do not ablate individual geometry factors, we do not attribute this ranking to a specific geometric property.

In contrast, the current policy performs poorly on the rectangular nut. A common failure mode occurs after the nut is turned 60° clockwise: in the next cycle, the fingers fail to establish contact with the next pair of faces, leaving the nut stuck at the first 60° position. A smaller nut such as the M24 can escape this failure mode by chance, but it survives for only another two cycles before the mismatched policy completely fails. Together, these results suggest that transfer is strong when an unseen profile preserves cyclic re-grasp configurations compatible with training, but degrades sharply when its face topology conflicts with the learned re-grasp strategy.

V. Conclusion and Future Work

A. Conclusion

We presented NUTS, a framework for learning long-horizon cyclic nut threading with dexterous hands. Rather than learning the entire task with a monolithic policy, NUTS decomposes demonstration generation into an RL-learned, goal-conditioned turning skill and scripted transition behaviors, while distilling the resulting long-horizon trajectories into a unified visuomotor policy for deployment. Human corrections during real-world replay further adapt the demonstration distribution to contact and perception discrepancies that are difficult to capture in simulation.

Experiments show that the distilled policy reliably completes 10 consecutive threading cycles on all training-distribution nut sizes and transfers to unseen round and octagonal profiles, while rectangular profiles expose a limitation of the learned re-grasp strategy. These results suggest that structured behavior generation followed by unified policy distillation provides an effective approach to long-horizon, contact-rich dexterous manipulation.

B. Future Work

Long-horizon stress tests reveal a clear limitation: the current student policy eventually fails after 15–22 cycles, depending on nut size, geometry, and test conditions. This degradation appears to be largely due to limited domain randomization along the z-axis, which inadequately covers the axial drift accumulated over many cycles. An important next

step is therefore to introduce a curriculum that progressively expands the axial variation encountered during training, improving the long-horizon capability of the RL teacher and, consequently, the distilled student.

To simplify the task, we deliberately initialize the nut already engaged with the bolt. We adopted this setting after unsuccessful attempts to learn or demonstrate nut engagement using reinforcement learning, teleoperation, and imitation learning. We attribute much of the difficulty to the rigid nut and the helical thread constraint, which admit only a narrow entry configuration for successful engagement. Future work will therefore focus on autonomous nut engagement, which we anticipate will require fine-grained fingertip-force feedback and tactile sensing.

REFERENCES

[1] Y. Narang, K. Storey, I. Akinola, M. Macklin, P. Reist, L. Wawrzyniak, Y. Guo, A. Moravanszky, G. State, M. Lu, A. Handa, and D. Fox, "Factory: Fast contact for robotic assembly," in *Proceedings of Robotics: Science and Systems*, New York City, NY, USA, June 2022. [Online]. Available: https://www.roboticsproceedings.org/rss18/p035.html

[2] M. Macklin, K. Erleben, M. Müller, N. Chentanez, S. Jeschke, and Z. Corse, "Local optimization for robust signed distance field collision," *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 3, no. 1, pp. 8:1–8:17, May 2020.

[3] E. Hsieh, W.-H. Hsieh, Y.-J. Wang, T. Lin, J. Malik, K. Sreenath, and H. Qi, "Learning dexterous manipulation skills from imperfect simulations," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026, arXiv:2512.02011. [Online]. Available: https://dexscrew.github.io/

[4] T. G. W. Lum, K. Kedia, C. K. Liu, and J. Bohg, "Play2Perfect: What matters in dexterous play pretraining for precise assembly?" *arXiv preprint arXiv:2606.26428*, 2026. [Online]. Available: https://arxiv.org/abs/2606.26428

[5] K. Kedia, T. G. W. Lum, J. Bohg, and C. K. Liu, "SimToolReal: An object-centric policy for zero-shot dexterous tool manipulation," *arXiv preprint arXiv:2602.16863*, 2026. [Online]. Available: https://arxiv.org/abs/2602.16863

[6] B. Siciliano and O. Khatib, Eds., *Springer Handbook of Robotics*, 2nd ed. Cham, Switzerland: Springer, 2016.

[7] M. Noseworthy, B. Tang, B. Wen, A. Handa, C. C. Kessens, N. Roy, D. Fox, F. Ramos, Y. Narang, and I. Akinola, "FORGE: Force-guided exploration for robust contact-rich manipulation under uncertainty," *IEEE Robotics and Automation Letters*, vol. 10, no. 5, pp. 4436–4443, 2025.

[8] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 15. PMLR, 2011, pp. 627–635. [Online]. Available: https://proceedings.mlr.press/v15/ross11a.html

[9] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[10] Y. Lee, J. J. Lim, A. Anandkumar, and Y. Zhu, "Adversarial skill chaining for long-horizon robot manipulation via terminal state regularization," in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 406–416. [Online]. Available: https://proceedings.mlr.press/v164/lee22a.html

[11] Y. Chen, C. Wang, L. Fei-Fei, and C. K. Liu, "Sequential dexterity: Chaining dexterous policies for long-horizon manipulation," in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 3809–3829. [Online]. Available: https://proceedings.mlr.press/v229/chen23e.html

[12] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. [Online]. Available: https://ieeexplore.ieee.org/document/8202133

[13] A. Wilcox, M. Ghanem, M. Moghani, P. Barroso, B. Joffe, and A. Garg, "Adapt3R: Adaptive 3D scene representation for domain transfer in imitation learning," in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 1486–1514. [Online]. Available: https://proceedings.mlr.press/v305/wilcox25a.html

[14] Y. Jiang, C. Wang, R. Zhang, J. Wu, and L. Fei-Fei, "TRANSIC: Sim-to-real policy transfer by learning from online correction," in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 1691–1729. [Online]. Available: https://proceedings.mlr.press/v270/jiang25a.html

[15] J. Zhang and K. Cho, "Query-efficient imitation learning for end-to-end autonomous driving," *arXiv preprint arXiv:1605.06450*, 2016. [Online]. Available: https://arxiv.org/abs/1605.06450

[16] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by cheating," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100. PMLR, 2020, pp. 66–75. [Online]. Available: https://proceedings.mlr.press/v100/chen20a.html

[17] R. Singh, A. Allshire, A. Handa, N. Ratliff, and K. V. Wyk, "DextrAH-RGB: Visuomotor policies to grasp anything with dexterous hands," *arXiv preprint arXiv:2412.01791*, 2024. [Online]. Available: https://arxiv.org/abs/2412.01791

[18] W. Wang, F. Wei, L. Zhou, X. Chen, L. Luo, X. Yi, Y. Zhang, Y. Liang, C. Xu, Y. Lu, J. Yang, and B. Guo, "UniGraspTransformer: Simplified policy distillation for scalable dexterous robotic grasping," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 12 199–12 208. [Online]. Available: https://dexhand.github.io/UniGraspTransformer/

[19] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. M. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. [Online]. Available: https://www.roboticsproceedings.org/rss19/p026.html

[20] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://openreview.net/forum?id=PqvMRDCJT9t

[21] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, L. Smith, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, "π_0: A vision-language-action flow model for general robot control," in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. [Online]. Available: https://www.roboticsproceedings.org/rss21/p010.html

[22] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations," in *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024. [Online]. Available: https://www.roboticsproceedings.org/rss20/p067.html

[23] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.